

Jupyter Notebook Execution Report

Name: Your Name

Date: April 04, 2026

Cell 1: ■ Markdown

■ **Análisis exploratorio: ■■■→■ Life Style Data**

1■■ ****Entender el contexto del proyecto****

El análisis exploratorio que se va a realizar es sobre el dataset ****Life Style Data****, el cuál puedes encontrar en el siguiente enlace:

■<https://www.kaggle.com/datasets/jockeroika/life-style-data>

■ ****Objetivo del análisis****

El propósito de este análisis es ****explorar y comprender cómo los hábitos de vida, la alimentación y la actividad física influyen en distintos indicadores de salud y rendimiento físico****.

A través de un análisis exploratorio detallado, se pretende:

- Identificar patrones generales en los datos relacionados con el estilo de vida de los participantes.
- Evaluar la relación entre variables como la frecuencia de ejercicio, la dieta, el IMC y el consumo calórico.
- Detectar posibles valores atípicos o inconsistencias en los datos que puedan afectar el modelado posterior.
- Preparar el dataset para aplicar modelos predictivos que permitan ****estimar indicadores de salud o rendimiento físico**** (por ejemplo, las calorías quemadas o el porcentaje de grasa corporal).

Además, en etapas posteriores se implementarán técnicas de ****regresión lineal (simple y múltiple)****, ****regularización (Ridge y Lasso)****, y ****modelos de aprendizaje automático basados en árboles de decisión y ensambles (Random Forest y Gradient Boosting)**** para comparar su rendimiento y capacidad predictiva.

En conjunto, este proyecto busca ****extraer información útil y visualmente comprensible**** sobre los factores que contribuyen a un estilo de vida saludable, apoyando la toma de decisiones y el diseño de rutinas personalizadas.

■ ****Significado de las variables****

Columna	Descripción
`Age`	Edad del participante (en años).
`Gender`	Género biológico (Masculino/Femenino).
`Weight (kg)`	Peso del individuo en kilogramos.
`Height (m)`	Altura del individuo en metros.
`Max_BPM`	Frecuencia cardíaca máxima registrada durante una sesión de ejercicio.
`Avg_BPM`	Frecuencia cardíaca promedio mantenida durante la sesión.
`Resting_BPM`	Frecuencia cardíaca en reposo antes de iniciar el entrenamiento.
`Session_Duration (hours)`	Duración de la sesión de ejercicio, en horas.
`Calories_Burned`	Total de calorías quemadas durante la sesión.
`Workout_Type`	Tipo de entrenamiento realizado (por ejemplo: fuerza, HIIT, cardio).
`Fat_Percentage`	Porcentaje de grasa corporal del individuo.
`Water_Intake (liters)`	Promedio diario de consumo de agua, en litros.
`Workout_Frequency (days/week)`	Número de días de entrenamiento por semana.
`Experience_Level`	Nivel de experiencia física (1 = Principiante, 2 = Intermedio, 3 = Avanzado).
`BMI`	Índice de Masa Corporal (IMC), medida de grasa corporal basada en peso y altura.
`Daily meals frequency`	Número de comidas consumidas diariamente.
`Physical exercise`	Indica el tipo o la frecuencia de la actividad física realizada.
`Carbs`	Consumo diario de carbohidratos (en gramos).
`Proteins`	Consumo diario de proteínas (en gramos).
`Fats`	Consumo diario de grasas (en gramos).
`Calories`	Total de calorías ingeridas a través de la alimentación diaria.
`meal_name`	Nombre de la comida (por ejemplo: desayuno, almuerzo, cena).
`meal_type`	Tipo de comida (por ejemplo: snack, plato principal, bebida).
`diet_type`	Tipo de dieta seguida (por ejemplo: cetogénica, vegana, equilibrada).
`sugar_g`	Contenido de azúcar en gramos por comida.
`sodium_mg`	Contenido de sodio en miligramos por comida.
`cholesterol_mg`	Contenido de colesterol en miligramos por comida.
`serving_size_g`	Tamaño de la porción del alimento en gramos.

| `cooking\_method` | Método de cocción utilizado (por ejemplo: hervido, frito, a la parrilla). |

| `prep\_time\_min` | Tiempo de preparación de la comida, en minutos. |

| `cook\_time\_min` | Tiempo de cocción de la comida, en minutos. |

| `rating` | Valoración de la comida o entrenamiento (escala de 1 a 5). |

| `Name of Exercise` | Nombre del ejercicio realizado. |

| `Sets` | Número de series completadas en el ejercicio. |

| `Reps` | Número de repeticiones por serie. |

| `Benefit` | Descripción del beneficio físico del ejercicio. |

| `Burns Calories (per 30 min)` | Calorías estimadas que se queman en 30 minutos de ese ejercicio. |

| `Target Muscle Group` | Grupo muscular principal trabajado en el ejercicio. |

| `Equipment Needed` | Equipamiento necesario para realizar el ejercicio. |

| `Difficulty Level` | Nivel de dificultad del ejercicio (Principiante, Intermedio, Avanzado). |

| `Body Part` | Parte del cuerpo principalmente involucrada (por ejemplo: brazos, piernas, pecho). |

| `Type of Muscle` | Tipo de músculo implicado (por ejemplo: superior, core, fuerza de agarre). |

| `Workout` | Nombre específico del entrenamiento o ejercicio. |

■ *Fuente de los datos: Kaggle - Global Coffee Health Dataset (2024)*

Cell 2: ■ Markdown

Lo primero de todo, será importar todas las librerías que vayamos a usar para realizar el análisis

Cell 3: ■ Code

```
import warnings
from scipy import stats
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.linear_model import Lasso, LassoCV, LinearRegression, Ridge, RidgeCV
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor, plot_tree
warnings.filterwarnings('ignore')
```

Cell 4: ■ Markdown

2.2.2 **Análisis descriptivo inicial**

Antes de comenzar con el análisis y la limpieza de datos, realizamos un ****análisis descriptivo inicial**** del dataset con el objetivo de conocer su estructura, tamaño y el tipo de información que contiene.

Este paso nos permite identificar el número de registros disponibles, las columnas existentes y obtener una primera impresión sobre la distribución general de los datos.

Cell 5: ■ Code

```
# Cargar el dataset
df = pd.read_csv("data/life_style_data.csv")
```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "<string>", line 1
    df = pd.read_csv("data/life_style_data.csv")
        ^
SyntaxError: invalid syntax
During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
    exec(source, glb)
    ~~~~^
  File "<string>", line 2, in <module>
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf\pdf.py", line 10
    return _read(filepath_or_buffer, kwds)
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf\pdf.py", line 10
    parser = TextFileReader(filepath_or_buffer, **kwds)
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf\pdf.py", line 10
    self._engine = self._make_engine(f, self.engine)
                    ~~~~~^
  File "c:\Users\rotap\aaoprod\aprendizaje\bigdata_AHK\PythonDataAnalysis_bigdata_AHK\.venv\Lib\site-packages\nb2pdf\pdf.py", line 10
    self.handles = get_handle(

```


Output:

```
■ Primeras 5 filas del dataset:
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 10: ■ Markdown

Por último, mostramos las estadísticas básicas de cada columna (cantidad total de filas, mediana, mínimo, máximo, 25%, 50%, 70%)

Cell 11: ■ Code

```
print("\n■ Estadísticas descriptivas:\n")
print(df.describe())
```

Output:

```
■ Estadísticas descriptivas:
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 12: ■ Markdown

3■■ ****Detección y tratamiento datos ausentes****

Antes de proceder con el análisis exploratorio y la creación de nuevas variables, es fundamental ****verificar la existencia de valores nulos o ausentes**** en el dataset.

La presencia de datos faltantes puede afectar la precisión de los resultados, alterar las estadísticas descriptivas y comprometer el rendimiento de los modelos predictivos.

Por ello, en esta sección se realiza una revisión general para comprobar si existen columnas o registros con valores nulos que deban ser tratados o imputados.

Cell 13: ■ Code

```
total_nulos = df.isnull().sum()
print(total_nulos)
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 14: ■ Markdown

Tras ejecutar el análisis, se observa que ninguna de las columnas contiene valores nulos.

Esto indica que el dataset está completo y no requiere imputación ni eliminación de registros, por lo que se puede continuar con el análisis exploratorio sin necesidad de realizar tratamientos adicionales sobre los datos faltantes.

Cell 15: ■ Markdown

4■■■ ****Revisar los tipos de las variables****

Una vez verificada la integridad de los datos, el siguiente paso consiste en ****revisar los tipos de las variables**** para asegurarnos de que cada columna tenga el formato adecuado según el tipo de información que representa.

Este proceso es fundamental para evitar errores en los cálculos posteriores y garantizar que las operaciones estadísticas o de modelado se apliquen correctamente.

Cell 16: ■ Code

```
# Mostramos un resumen de los tipos de datos de cada columna
print(df.info())
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403
    result = eval(lines[-1], glb)
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 17: ■ Markdown

En este caso, hemos detectado algunas columnas que con formato decimal cuando deberían ser un número entero, por lo que convertiremos esas columnas para el correcto tratamiento de los datos.

Las columnas que vamos a convertir a número entero son: `Workout_Frequency`, `Experience_level` y `Daily meals frequency`.

El motivo para convertir estas columnas entero, son que, por ejemplo en el caso de `Workout_Frequency`, lo que representa es el número de días que la persona entrena por semana, no tiene sentido entrenar 3.6 días a la semana.

Por otro lado, en el caso de `Experience_Level` lo que representa es el nivel de experiencia física (1 = Principiante, 2 = Intermedio, 3 = Avanzado), son etiquetas numéricas, por lo que no tiene sentido que sean float.

Cell 18: ■ Code

```
df['Workout_Frequency'] = df['Workout_Frequency (days/week)'].astype(int)
df['Experience_Level'] = df['Experience_Level'].astype(int)

df.head()
```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 1, in <module>

NameError: name 'df' is not defined

#### Cell 19: ■ Markdown

## 5 ■ ■ \*\*Identificación y tratamiento de datos atípicos\*\*

#### Cell 20: ■ Markdown

En esta sección se lleva a cabo la identificación y análisis de posibles valores atípicos presentes en el dataset. Los outliers pueden deberse a errores de registro, inconsistencias en la carga de datos o simplemente reflejar casos extremos pero reales dentro de la población.

El objetivo es verificar la coherencia y plausibilidad de los datos, asegurando que las observaciones sean consistentes con la realidad del contexto físico y nutricional que se analiza.

Para este proceso se utilizó principalmente el método `describe()`, que permite obtener estadísticas descriptivas básicas como los valores `mínimos`, `máximos`, `medias`, `medianas` y `percentiles`. Gracias a ello, se pudieron detectar valores que inicialmente podrían parecer extremos.

No obstante, en lugar de eliminarlos directamente, se realizó una revisión contextual comparando variables relacionadas —por ejemplo, la relación entre el peso, la altura y las calorías quemadas durante una sesión— para determinar si los valores eran coherentes desde un punto de vista

fisiológico.

Este enfoque permite distinguir entre errores reales y casos válidos, evitando eliminar información relevante y manteniendo la integridad del dataset para el análisis posterior.

#### Cell 21: ■ Code

```
# Usamos la función describe para evaluar de forma rápida si hay algún valor atípico en nuestro dataset  
df.describe()
```

#### Error:

```
Traceback (most recent call last):  
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 403  
    result = eval(lines[-1], glb)  
  File "<string>", line 1, in <module>  
NameError: name 'df' is not defined
```

#### Cell 22: ■ Markdown

##### Verificación de la fiabilidad de los datos dietéticos

Antes de analizar la relación entre la dieta y la composición corporal de los deportistas, es fundamental comprobar que los datos dietéticos sean **fiables y coherentes**.

En primer lugar, verificaremos si las calorías calculadas a partir de los macronutrientes coinciden con las calorías declaradas en la columna `Calories`.

Posteriormente, analizaremos si los porcentajes de macronutrientes (`Carb\_ratio`, `Protein\_ratio`, `Fat\_ratio`) reflejan adecuadamente los valores esperados para cada tipo de dieta (`Diet\_Type`).

#### Cell 23: ■ Markdown

##### Comprobación del balance energético de macronutrientes

Para estimar las calorías teóricas derivadas de los macronutrientes, se utilizan equivalencias nutricionales ampliamente aceptadas:

\* **1 g de proteína = 4 kcal**

\* **1 g de carbohidrato = 4 kcal**

\* **1 g de grasa = 9 kcal**

#### Cell 24: ■ Code

```
calorias_macro = df['Carbs'] * 4 + df['Proteins'] * 4 + df['Fats'] * 9  
calorias_macro
```

**Error:**

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

**Cell 25: ■ Markdown**

A continuación, se evalúa la discrepancia entre las calorías calculadas y las registradas en la columna `Calories`.

Se considera aceptable una diferencia máxima del  $\pm 5\%$ , representada por la línea roja en la siguiente figura.

**Cell 26: ■ Code**

```
dif = df['Calories'] - calorias_macro

pct_discrepancia = dif / calorias_macro * 100

plt.figure(figsize=(10,6))
sns.histplot(pct_discrepancia.abs(), bins=60, kde=True, color='C0',
edgecolor='white', alpha=0.8)
plt.axvline(5, color='red', linestyle='--', linewidth=2)
plt.xlabel('Porcentaje de discrepancia (%)')
plt.ylabel('Frecuencia')
plt.title('Distribución del % discrepancia entre Calories y calorías desde
macronutrientes')
plt.grid(alpha=0.3)
plt.show()
```

**Error:**

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

**Cell 27: ■ Markdown**

Los resultados muestran que **la mayoría de los registros presentan discrepancias superiores al 5%**, lo cual indica que la columna `Calories` **no es consistente** con los valores de macronutrientes.

Esto sugiere errores en la generación o registro de los datos, por lo que **las calorías totales no pueden considerarse una medida fiable** del valor energético real derivado de proteínas, carbohidratos y grasas.

#### Cell 28: ■ Markdown

#### Análisis de la coherencia interna de los macronutrientes

Para determinar si el problema proviene de los datos de macronutrientes, se calculan los porcentajes energéticos que aporta cada uno sobre el total:

#### Cell 29: ■ Code

```
macro_kcal = (df['Carbs'] * 4) + (df['Proteins'] * 4) + (df['Fats'] * 9)

Carb_ratio = (df['Carbs'] * 4 / macro_kcal).round(1)
Protein_ratio = (df['Proteins'] * 4 / macro_kcal).round(1)
Fat_ratio = (df['Fats'] * 9 / macro_kcal).round(1)

print(f"Lineas donde los carbohidratos son el 50% de la ingesta: {(Carb_ratio == 0.5).mean() * 100}%")

print(f"Lineas donde las proteinas son el 20% de la ingesta: {(Protein_ratio == 0.2).mean() * 100}%")

print(f"Lineas donde las grasas son el 30% de la ingesta: {(Fat_ratio == 0.3).mean() * 100}%")
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
 File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

#### Cell 30: ■ Markdown

Los resultados muestran que **una gran proporción de registros mantienen exactamente las mismas proporciones (50%-20%-30%)**, independientemente del tipo de dieta (`keto`, `low carb` o `balanced`).

Este patrón es **poco realista** y **contradice los valores nutricionales esperados**, donde una dieta cetogénica, por ejemplo, debería tener una proporción mucho menor de carbohidratos y mayor de grasas.

Por tanto, se concluye que tanto los campos de macronutrientes como ``Diet_Type`` presentan **\*\*problemas de consistencia\*\*** y **\*\*no reflejan adecuadamente la información dietética que pretenden representar\*\***.

#### Cell 31: ■ Markdown

El análisis de coherencia revela que:

- \* Las **\*\*calorías derivadas de los macronutrientes no coinciden\*\*** con las calorías totales declaradas.
- \* Los **\*\*porcentajes de macronutrientes son prácticamente iguales\*\*** entre dietas teóricamente distintas.

En consecuencia, **\*\*las columnas ``Calories``, ``Carbs``, ``Proteins``, ``Fat`` y ``Diet_Type`` presentan inconsistencias\*\*** que limitan su utilidad para un análisis predictivo fiable.

#### Cell 32: ■ Markdown

Si analizamos los datos que nos reporta el método ``describe()``, observamos algunos valores que en un primer momento podrían parecer atípicos, como el máximo de calorías quemadas por sesión (``Calories_Burned`` = 2890.82 kcal).

Sin embargo, al revisar conjuntamente las columnas de altura y peso, comprobamos que los valores máximos registrados (``Height`` = 2.01 m, ``Weight`` = 130.77 kg) corresponden a individuos con una complexión grande.

Esto tiene sentido desde el punto de vista fisiológico, ya que una persona más alta y con mayor masa corporal consume más energía durante el ejercicio, incluso realizando la misma actividad que alguien más pequeño.

Además, el análisis de la variable ``Experience_Level`` muestra que la mayoría de participantes se encuentran entre los niveles 1 y 2 (principiante e intermedio), lo que sugiere que el gasto calórico elevado puede deberse también a menor eficiencia en la ejecución del ejercicio, ya que las personas con menos experiencia tienden a gastar más energía para mantener la misma intensidad.

Por tanto, estos valores altos de calorías no se consideran errores ni outliers anómalos, sino variaciones naturales explicadas por las características físicas y el nivel de experiencia de los individuos.

---

#### Cell 33: ■ Markdown

Durante la exploración inicial del dataset, se identificaron algunas columnas cuyos valores no coincidían con la descripción oficial proporcionada por la fuente del conjunto de datos, o bien no aportaban información relevante para el análisis.

Este proceso de revisión tiene como objetivo garantizar la calidad y consistencia semántica de las variables, eliminando aquellas que puedan inducir ruido o confusión en los resultados posteriores.

En concreto:

- La columna `meal\_name` se eliminó porque todos sus valores corresponden al término "Other", a pesar de que la descripción oficial indica que debería contener nombres de comidas como Breakfast, Lunch o Dinner. Al no aportar variabilidad ni información útil, esta variable se descartó.

- La columna `Physical exercise` fue removida tras comprobar que sus valores numéricos (por ejemplo, -0.04, 0.97, 3.0) no se ajustaban a la descripción original ("indica el tipo o la frecuencia de actividad física"), lo que sugiere un error en la codificación o carga de datos.

Estas eliminaciones permiten mantener la coherencia entre el significado teórico de las variables y los datos reales presentes en el dataset, mejorando la fiabilidad del análisis posterior.

#### Cell 34: ■ Code

```
print((f"Total columnas antes de la limpieza: {len(df.columns)}"))

Creamos un nuevo DataFrame sin las columnas que no son útiles
df = df.drop(['meal_name', 'Physical exercise', 'Calories', 'Carbs', 'Proteins',
'Fats', 'diet_type', 'Daily meals frequency', 'sugar_g', 'sodium_mg',
'cholesterol_mg', 'serving_size_g', 'cooking_method', 'prep_time_min',
'cook_time_min', 'rating', 'protein_per_kg', 'cal_balance', 'cal_from_macros',
'pct_carbs', 'meal_type'], axis=1)

Mostramos de nuevo las 4 primeras filas para verificar que está todo correcto
print((f"Total columnas después de la limpieza: {len(df.columns)}"))
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

#### Cell 35: ■ Markdown

### 6■■■ **\*\*Creación de variables derivadas o auxiliares\*\***

#### Cell 36: ■ Markdown

Tras la fase de limpieza y verificación de datos, hemos decidido **\*\*crear una serie de variables auxiliares\*\*** para enriquecer el análisis y obtener una visión más completa sobre los factores relacionados con el **\*\*rendimiento físico y la condición corporal\*\*** de los participantes.

El objetivo principal de este apartado es **\*\*aprovechar la información ya existente\*\*** en el dataset para generar nuevos indicadores que nos ayuden a interpretar mejor los resultados, detectar patrones y facilitar el análisis de correlaciones en fases posteriores.

Estas variables combinan datos de distintas columnas originales y nos permiten comparar de manera más justa el desempeño entre personas con características físicas diferentes. A continuación, se describen las nuevas variables creadas:

- **HRR (Heart Rate Reserve)**: calcula la **reserva de frecuencia cardíaca**, es decir, la diferencia entre la frecuencia máxima y la frecuencia en reposo ( $\text{Max\_BPM} - \text{Resting\_BPM}$ ). Este valor refleja la capacidad cardiovascular de cada individuo: cuanto mayor sea el HRR, mejor condición física suele tener la persona.
- **Relative intensity**: mide el **nivel de esfuerzo relativo** durante la sesión utilizando la **fórmula de Karvonen**, un método ampliamente utilizado para calcular la intensidad del ejercicio en función de la frecuencia cardíaca individual. Se obtiene mediante la expresión  $(\text{Avg\_BPM} - \text{Resting\_BPM}) / \text{HRR}$ , donde  $\text{HRR}$  representa la reserva de frecuencia cardíaca. Este valor refleja el **porcentaje de la capacidad cardiovascular utilizada** durante la sesión: cuanto más alto es el resultado, mayor es la intensidad del esfuerzo realizado en relación con el máximo potencial del individuo.
- **Calories/hour**: representa la **intensidad del entrenamiento** expresada en calorías quemadas por hora ( $\text{Calories\_Burned} / \text{Session\_Duration (hours)}$ ). Esta métrica permite comparar el esfuerzo entre sesiones de distinta duración de manera más equitativa.
- **Weekly training load**: estima la **carga total de entrenamiento semanal**, combinando la frecuencia y la duración media de las sesiones ( $\text{Workout\_Frequency} * \text{Session\_Duration}$ ). Cuanto mayor sea este valor, mayor es el volumen de entrenamiento acumulado en la semana.
- **Approximate energy balance**: mide el **balance energético diario aproximado** restando las calorías quemadas de las calorías ingeridas ( $\text{Calories} - \text{Calories\_Burned}$ ). Un valor positivo indica superávit calórico (ingesta mayor al gasto), mientras que un valor negativo indica déficit calórico.

La creación de estas variables nos permite **profundizar en el análisis exploratorio y predictivo**, ofreciendo una visión más contextualizada del rendimiento físico y los hábitos de los participantes. Gracias a estos nuevos indicadores, podremos identificar relaciones más precisas entre el entrenamiento, la dieta y los resultados fisiológicos.

### Cell 37: ■ Code

```
# Reserva de frecuencia cardíaca (FCR)
df['HRR'] = df['Max_BPM'] - df['Resting_BPM']

# Intensidad relativa
# Usamos la fórmula de Karvonen que es un método eficaz para calcular nuestras zonas
# de entrenamiento por frecuencia cardíaca
df['Relative_Intensity'] = (df['Avg_BPM'] - df['Resting_BPM']) / df['HRR']

# Calorías por hora (intensidad de entrenamiento)
df['Calories_per_hour'] = df['Calories_Burned'] / df['Session_Duration
(hours)'].replace(0, np.nan)

# Carga de entrenamiento semanal
df['Weekly_training_load'] = df['Workout_Frequency (days/week)'] *
df['Session_Duration (hours)']
```

```
# Verificación rápida de las nuevas columnas
df[['HRR', 'Relative_Intensity', 'Calories_per_hour',
'Weekly_training_load']].head()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
```

```
File "<string>", line 15
```

```
df[['HRR', 'Relative_Intensity', 'Calories_per_hour',
 ^
```

SyntaxError: '[' was never closed

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 408
 exec(source, glb)
    ~~~~^
```

```
File "<string>", line 2, in <module>
```

NameError: name 'df' is not defined

#### Cell 38: ■ Markdown

### Validación de Variables Derivadas

Se implementó un proceso de validación para verificar la coherencia de las nuevas variables:

#### Cell 39: ■ Code

```
print("\n■ Validación de nuevas columnas:\n")
```

```
# 1. HRR debe ser positivo
```

```
invalid_hrr = df[df['HRR'] <= 0]
```

```
print(f"\n1. HRR (Heart Rate Reserve):")
```

```
print(f" Registros con HRR ≤ 0: {len(invalid_hrr)}
({len(invalid_hrr)/len(df)*100:.2f}%)")
```

```
# 2. Relative_Intensity debe estar entre 0 y 1
```

```
invalid_intensity = df[(df['Relative_Intensity'] < 0) |
(df['Relative_Intensity'] > 1)]
```

```
print(f"\n2. Relative_Intensity:")
```

```
print(f" Registros fuera de rango [0,1]: {len(invalid_intensity)}
({len(invalid_intensity)/len(df)*100:.2f}%)")
```



#### Cell 42: ■ Markdown

En esta sección se analizan las relaciones entre distintas variables del dataset con el objetivo de responder a preguntas específicas sobre la salud, el rendimiento físico y los hábitos de los participantes en este dataset.

#### Cell 43: ■ Markdown

### ■ **Bloque A: Salud Cardiovascular y Composición Corporal**

El ritmo cardíaco en reposo (`Resting\_BPM`) y la reserva cardíaca (`HRR`) son indicadores clave de salud cardiovascular. Este bloque compara el poder predictivo de dos grupos de variables mediante **regresión lineal múltiple**:

#### Cell 44: ■ Markdown

#### **Pregunta 1:** ¿Cómo se relaciona el `Fat\_Percentage` con el `HRR`? ¿El nivel de experiencia modifica esta relación?

#### Cell 45: ■ Code

```
# Correlación simple
corr = df['Fat_Percentage'].corr(df['HRR'])
print(f"Correlación Fat_Percentage vs HRR: {corr:.3f}")

# Visualización general
sns.regplot(
    data=df,
    x='Fat_Percentage',
    y='HRR',
    scatter_kws={'alpha': 0.4},
    line_kws={'color': 'red'}
)
plt.title(f'Relación entre porcentaje de grasa corporal y HRR (r = {corr:.2f})')
plt.xlabel('Fat Percentage (%)')
plt.ylabel('Heart Rate Reserve (HRR)')
plt.show()

# Visualización por nivel de experiencia
```

```
sns.lmplot(
data=df,
x='Fat_Percentage',
y='HRR',
hue='Experience_Level',
palette='Set2',
height=6,
aspect=1.3
)
plt.title('Relación entre % de grasa y HRR según nivel de experiencia')
plt.xlabel('Fat Percentage (%)')
plt.ylabel('HRR')
plt.show()
```

#### Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

#### Cell 46: ■ Markdown

Podemos ver que el porcentaje de grasa no esta relacionado con la reserva cardiaca de ninguna forma pero sí vemos que los deportistas mas experimentados tienen menor porcentaje de grasa

#### Cell 47: ■ Markdown

#### \*\*Pregunta 2:\*\* ¿Las personas con menor nivel de experiencia tienen un ritmo cardíaco en reposo (Resting\_BPM) más alto y una reserva cardíaca (HRR) más baja que las más experimentadas?

#### Cell 48: ■ Code

```
RESUMEN DE MEDIAS POR NIVEL DE EXPERIENCIA
summary = df.groupby('Experience_Level')[['Resting_BPM', 'HRR']].mean().round(2)
print(summary)
```

```
VISUALIZACIÓN CON GRÁFICOS DE BARRAS
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
sns.barplot(
data=df,
x='Experience_Level',
y='Resting_BPM',
palette='Reds',
ci='sd',
ax=axes[0]
)
axes[0].set_title('Ritmo cardíaco en reposo por nivel de experiencia',
fontweight='bold')
axes[0].set_xlabel('Nivel de experiencia (1=Principiante, 3=Avanzado)')
axes[0].set_ylabel('Resting BPM')
```

```
sns.barplot(
data=df,
x='Experience_Level',
y='HRR',
palette='Greens',
ci='sd',
ax=axes[1]
)
axes[1].set_title('HRR (Reserva cardíaca) por nivel de experiencia',
fontweight='bold')
axes[1].set_xlabel('Nivel de experiencia (1=Principiante, 3=Avanzado)')
axes[1].set_ylabel('HRR')
```

```
plt.tight_layout()
plt.show()
```

#### Error:

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in <module>
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

#### Cell 49: ■ Code

```
# DISTRIBUCIÓN INDIVIDUAL CON BOXPLOTS
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.boxplot(
    data=df,
    x='Experience_Level',
    y='Resting_BPM',
    palette='Reds',
    ax=axes[0]
)

axes[0].set_title('Distribución de Resting BPM por nivel de experiencia',
    fontweight='bold')
axes[0].set_xlabel('Nivel de experiencia')
axes[0].set_ylabel('Resting BPM')

sns.boxplot(
    data=df,
    x='Experience_Level',
    y='HRR',
    palette='Greens',
    ax=axes[1]
)

axes[1].set_title('Distribución de HRR por nivel de experiencia',
    fontweight='bold')
axes[1].set_xlabel('Nivel de experiencia')
axes[1].set_ylabel('HRR')

plt.tight_layout()
plt.show()
```

#### Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 5, in <module>
NameError: name 'df' is not defined
```

#### Cell 50: ■ Markdown

No hay evidencia de que el nivel de experiencia se asocie con un Resting BPM más alto ni una HRR más baja en principiantes respecto a avanzados; las medias son prácticamente iguales y los intervalos de error se solapan. Esto probablemente se deba a que todos en la muestra son físicamente activos, por lo que su condición cardiovascular básica es similar independientemente del nivel de experiencia.

### Cell 51: ■ Markdown

Para complementar el análisis, se calcula una matriz de correlación centrada únicamente en las variables relacionadas con la salud cardiovascular y la composición corporal.

Esto permite identificar qué factores muestran una relación más fuerte con el ritmo cardíaco en reposo (Resting\_BPM) y la reserva cardíaca (HRR), así como explorar cómo la composición corporal (Fat\_Percentage, BMI) o el nivel de experiencia influyen en estos indicadores.

### Cell 52: ■ Code

```
Variables relevantes del bloque A
vars_bloque_a = [
 'Resting_BPM',
 'HRR',
 'Fat_Percentage',
 'BMI',
 'Experience_Level',
 'Workout_Frequency',
 'Relative_Intensity'
]

Filtrar las columnas que realmente existen en el DataFrame
vars_presentes = [v for v in vars_bloque_a if v in df.columns]
df_corr = df[vars_presentes].copy()

Calcular matriz de correlación
corr_matrix = df_corr.corr()

Visualización
plt.figure(figsize=(9, 6))
sns.heatmap(
 corr_matrix,
 annot=True,
 cmap='coolwarm',
```

```

center=0,
linewidths=0.5,
fmt=".2f"
)

plt.title('■ Correlación entre variables de salud cardiovascular y composición corporal', fontsize=13, fontweight='bold')

plt.tight_layout()

plt.show()

```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 13, in <module>

NameError: name 'df' is not defined

#### Cell 53: ■ Markdown

- Relación principal: Resting\_BPM muestra una correlación moderada-negativa con HRR. Interpretación: mayor reserva cardíaca ↔ menor frecuencia cardíaca en reposo (coherente fisiológicamente).
- Colinealidad alta: Fat\_Percentage y BMI muy correlacionados ( $\approx 0.90$ ). Indica redundancia — elegir una para modelos o usar regularización/PCA.
- Patrones de entrenamiento: Experience\_Level se asocia fuertemente con Workout\_Frequency — participantes más experimentados entrenan con mayor regularidad.
- Poco poder explicativo para Resting\_BPM fuera de HRR: otras variables (BMI, Fat\_Percentage, Workout\_Frequency, Relative\_Intensity) muestran correlaciones débiles con Resting\_BPM, por lo que gran parte de la variabilidad de Resting\_BPM queda sin explicar por este conjunto de features.

#### Cell 54: ■ Markdown

### ■ Bloque B: Rendimiento de sesión

#### Cell 55: ■ Markdown

**\*\*Objetivo:\*\*** Evaluar qué factores influyen en el gasto energético y la intensidad del entrenamiento.

**\*\*Contexto:\*\*** El rendimiento durante una sesión de ejercicio depende de múltiples variables, como el tipo de entrenamiento, la experiencia del individuo y su capacidad cardiovascular. Analizar estas relaciones permite identificar qué factores determinan una mayor quema de calorías, si los entrenamientos más intensos (como HIIT) se traducen realmente en un gasto calórico superior, y si los niveles de experiencia afectan la eficiencia del esfuerzo realizado.

## Cell 56: ■ Markdown

#### \*\*Pregunta 1:\*\* ¿Qué tipo de ejercicio tiende a quemar más calorías?

## Cell 57: ■ Code

```
sns.set(style="whitegrid", palette="muted")
plt.rcParams['figure.figsize'] = (12, 6)

Promedio de calorías por tipo de entrenamiento
mean_calories =
df.groupby('Workout_Type')['Calories_Burned'].mean().sort_values(ascending=False)
print(mean_calories)

Visualización
sns.barplot(
data=df,
x='Workout_Type',
y='Calories_Burned',
estimator='mean',
ci='sd',
palette='Set2'
)

plt.title('Calorías quemadas promedio por tipo de entrenamiento', fontsize=14,
fontweight='bold')
plt.xlabel('Tipo de entrenamiento')
plt.ylabel('Calorías quemadas (promedio)')
plt.xticks(rotation=30, ha='right')
plt.show()

sns.barplot(
data=df,
x='Workout_Type',
y='Calories_per_hour',
estimator='mean',
ci='sd',
```

```

palette='coolwarm'
)

plt.title('Intensidad promedio (Calorías/hora) por tipo de entrenamiento',
fontSize=14, fontWeight='bold')
plt.xlabel('Tipo de entrenamiento')
plt.ylabel('Calorías por hora (promedio)')
plt.xticks(rotation=30, ha='right')
plt.show()

```

#### Error:

```

Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 5, in <module>
NameError: name 'df' is not defined

```

#### Cell 58: ■ Markdown

- Resumen general: HIIT muestra la mayor intensidad (cal/h) y también las calorías totales más altas; Strength ocupa la segunda posición; Cardio queda en un rango intermedio; Yoga presenta la menor intensidad y los menores totales.
- Variabilidad: las barras de error indican dispersión importante dentro de cada tipo —hay sesiones HIIT o Strength con menor gasto y sesiones Cardio largas que consumen muchas calorías— por lo que no es determinante el tipo por sí solo.
- Relación intensidad vs total: las calorías totales resultan de intensidad × duración. HIIT combina alta intensidad y sesiones relativamente cortas, produciendo alto gasto por hora y alto total; en otros casos (p. ej. Strength o Cardio) un gasto total elevado puede deberse más a mayor duración o frecuencia que a más intensidad.

#### Cell 59: ■ Markdown

##### \*\*Pregunta 2:\*\* ¿Las personas con menos experiencia (`Experience\_Level`) queman más calorías por sesión que las avanzadas?

#### Cell 60: ■ Code

```

sns.barplot(
data=df,
x='Experience_Level',
y='Calories_Burned',

```

```

estimator='mean',
ci='sd',
palette='pastel'
)
plt.title('Calorías quemadas promedio según nivel de experiencia', fontsize=14,
fontweight='bold')
plt.xlabel('Nivel de experiencia (1 = Principiante, 3 = Avanzado)')
plt.ylabel('Calorías quemadas (promedio)')
plt.show()

```

```

sns.boxplot(
data=df,
x='Experience_Level',
y='Calories_Burned',
palette='Set3'
)
plt.title('Distribución de calorías quemadas por nivel de experiencia',
fontsize=14, fontweight='bold')
plt.xlabel('Nivel de experiencia')
plt.ylabel('Calorías quemadas')
plt.show()

```

```

summary = df.groupby('Experience_Level')[['Calories_Burned',
'Calories_per_hour']].mean().round(2)
print(summary)

```

#### Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 2, in <module>

NameError: name 'df' is not defined

Cell 61: ■ Markdown

Podemos ver que esta relacionada la experiencia con las calorías quemadas totales, pero no las quemadas por hora, lo cual nos indica que los que tienen mas experiencia entrenan mas horas:

Cell 62: ■ Code

```
if 'Experience_Level' in df.columns:
df['Experience_Level'] = pd.Categorical(df['Experience_Level'],
ordered=True,
categories=sorted(df['Experience_Level'].unique()))

# Resumen rápido
summary_dur = df.groupby('Experience_Level')['Session_Duration
(hours)'].agg(['mean', 'median', 'std', 'count']).round(2)
print("Resumen Session_Duration por Experience_Level:\n", summary_dur)

# Gráfica: distribución por sesión + medias
fig, axes = plt.subplots(1, 2, figsize=(14,5))

# Boxplot + swarm para ver distribución por nivel
sns.boxplot(data=df, x='Experience_Level', y='Session_Duration (hours)',
palette='pastel', ax=axes[0])
axes[0].set_title('Duración por sesión según nivel de experiencia')
axes[0].set_xlabel('Experience_Level')
axes[0].set_ylabel('Session_Duration (hours)')

# Barplot de la media (con desviación estándar)
sns.barplot(data=df, x='Experience_Level', y='Session_Duration (hours)',
estimator='mean', ci='sd', palette='muted', ax=axes[1])
axes[1].set_title('Duración media por sesión por Experience_Level')
axes[1].set_xlabel('Experience_Level')
axes[1].set_ylabel('Session_Duration (hours)')

plt.suptitle('Relación entre Experience_Level y duración de la sesión (hours)',
fontsize=14)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```

Error:

Traceback (most recent call last):

```
File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 63: ■ Markdown

```
##### **Matriz de correlación**
```

Cell 64: ■ Markdown

Para profundizar en el análisis del rendimiento de las sesiones, se calcula una matriz de correlación que incluye únicamente las variables más relevantes: gasto calórico (Calories_Burned), intensidad relativa (Relative_Intensity), frecuencia de entrenamiento (Workout_Frequency), experiencia (Experience_Level), y medidas cardiovasculares (HRR, Resting_BPM).

Este análisis permite identificar qué factores guardan una relación más fuerte con el gasto energético y la intensidad del entrenamiento, y si la experiencia o la condición cardiovascular influyen significativamente en el rendimiento observado.

Cell 65: ■ Code

```
# Definir las variables relevantes para el Bloque B
vars_bloque_b = [
    'Calories_Burned',
    'Calories_per_hour',
    'Workout_Frequency',
    'Relative_Intensity',
    'Experience_Level',
    'HRR',
    'Resting_BPM'
]

# Filtrar solo las columnas que existan en el DataFrame
vars_presentes = [v for v in vars_bloque_b if v in df.columns]
df_corr = df[vars_presentes].copy()

# Calcular la matriz de correlación
corr_matrix = df_corr.corr()

# Visualización con Seaborn
plt.figure(figsize=(10, 7))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
```

```

center=0,
linewidths=0.5,
fmt=".2f"
)

plt.title('■ Correlación entre variables de rendimiento de sesión', fontsize=14,
fontweight='bold')

plt.show()

```

Error:

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 13, in <module>

NameError: name 'df' is not defined

#### Cell 66: ■ Markdown

Relaciones principales:

- Calories\_Burned se relaciona fuertemente con Calories\_per\_hour y también con Experience\_Level y Workout\_Frequency . Interpretación: mayor volumen/intensidad y más experiencia/frecuencia se asocian a mayor gasto calórico total.
- Calories\_per\_hour y Calories\_Burned alto vínculo lógico (métrica de intensidad vs total).
- Experience\_Level y Workout\_Frequency están fuertemente correlacionados : participantes más experimentados entrenan con más frecuencia.

Implicaciones para modelado:

Para predecir Calories\_Burned usar Calories\_per\_hour, Workout\_Frequency y Experience\_Level puede mejorar desempeño.

#### Cell 67: ■ Markdown

### ■■ Bloque C: Perfil general de los participantes

**\*\*Objetivo:\*\*** Comprender el perfil dominante en el dataset y analizar las diferencias entre grupos.

**\*\*Contexto:\*\*** Explorar las variables demográficas y de estilo de vida permite identificar el perfil general de los individuos del dataset: si son más bien sedentarios o activos, si entrenan con regularidad y si su balance energético es positivo o negativo.

Además, observar las diferencias entre hombres y mujeres en cuanto a frecuencia de entrenamiento, tipo de ejercicios y gasto calórico puede revelar tendencias interesantes sobre los hábitos de entrenamiento y alimentación.

**\*\*Pregunta 1:\*\*** ¿Predominan sujetos con un estilo de vida activo o sedentario?

**\*\*Pregunta 2:\*\*** ¿Entrenan con más frecuencia los hombres o las mujeres? (`Gender` vs `Workout\_Frequency`)

**\*\*Pregunta 3:\*\*** ¿Qué tipo de ejercicio prefieren los hombres y las mujeres? (`Gender` vs `Workout\_Type`)

## Cell 68: ■ Markdown

### **\*\*1 — Clasificación de nivel de actividad\*\***

En primer lugar, definimos un umbral simple para distinguir entre participantes **\*\*activos\*\*** y **\*\*sedentarios\*\***, según su frecuencia semanal de entrenamiento (`Workout\_Frequency`).

Un valor superior a 2 días por semana se considera indicador de un estilo de vida activo.

A continuación, se calcula y representa la distribución de ambos grupos, tanto en términos absolutos como porcentuales, para identificar cuál predomina en el conjunto de datos.

Esta primera visualización permite tener una idea general del tipo de población que compone el dataset.

## Cell 69: ■ Code

```
# Definir umbral simple para considerar "activo" vs "sedentario"
activo_umbral = 2 # días/semana

df['Activity_level'] = np.where(df['Workout_Frequency'] > activo_umbral,
                                'Active', 'Sedentary')

# 1) ¿Predominan sujetos activos o sedentarios?
counts_activity = df['Activity_level'].value_counts()
pct_activity = df['Activity_level'].value_counts(normalize=True) * 100

print("Distribución Activity_level (conteo):")
print(counts_activity)
print("\nDistribución Activity_level (%):")
print(pct_activity.round(2))

plt.figure(figsize=(6,4))
sns.countplot(data=df, x='Activity_level', hue='Activity_level',
              order=['Active', 'Sedentary'])
plt.title('Conteo: Active vs Sedentary')
plt.xlabel('')
plt.ylabel('Número de sujetos')
plt.show()
```

**Error:**

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
 File "<string>", line 3, in <module>
NameError: name 'df' is not defined
```

**Cell 70: ■ Markdown**

Podemos ver que los "activos" (aquellos que entrenan 3 días o mas) predominan sobre los que entrenan 2 días o menos

**Cell 71: ■ Markdown****\*\*2 — Frecuencia de entrenamiento por género\*\***

En este bloque se analiza si existen diferencias notables en la frecuencia de entrenamiento entre hombres y mujeres.

Para ello, se agrupan los datos por `Gender` y se calculan medidas descriptivas como la **media**, **mediana** y **desviación estándar** de `Workout\_Frequency`.

Además, se utiliza un gráfico de barras para representar visualmente la media de días de entrenamiento por género, con intervalos de confianza al 95%.

Este análisis permite detectar si alguno de los géneros tiende a entrenar con mayor regularidad.

**Cell 72: ■ Code**

```
plt.figure(figsize=(8,4))
sns.barplot(data=df, x='Gender', y='Workout_Frequency', hue='Gender')
plt.title('Promedio de Workout_Frequency por Gender')
plt.ylabel('Días/semana (media)')
plt.show()
```

**Error:**

```
Traceback (most recent call last):
 File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
 exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

### Cell 73: ■ Markdown

Esta grafica perfectamente dividida nos hace pensar que los datos han sido creados de forma sintética o bien que se ha seleccionado a las personas del dataset.

### Cell 74: ■ Markdown

#### **\*\*3 — Tipos de entrenamiento preferidos por género\*\***

Aquí se explora la variable `Workout\_Type` para determinar los **tipos de ejercicio más comunes** y observar posibles diferencias de preferencia entre hombres y mujeres.

Se seleccionan los **Top 8** tipos más frecuentes y se construye un gráfico de barras agrupado por `Gender` para facilitar la comparación visual.

Este análisis ayuda a identificar tendencias de comportamiento y preferencias en el tipo de actividad física realizada, complementando la información sobre la frecuencia de entrenamiento.

### Cell 75: ■ Code

```
# 3) ¿Qué tipo de ejercicio prefieren hombres y mujeres?
# Mostrar los top N tipos y plot con hue Gender

top_n = 8

top_types = df['Workout_Type'].value_counts().head(top_n).index
df_types = df[df['Workout_Type'].isin(top_types)].copy()

plt.figure(figsize=(10,5))
sns.countplot(data=df_types, y='Workout_Type', hue='Gender', order=top_types,
palette='Set1')
plt.title(f'Top {top_n} Workout_Type por Gender')
plt.xlabel('Número de sujetos')
plt.ylabel('Workout_Type')
plt.legend(title='Gender')
plt.tight_layout()
plt.show()
```

#### **Error:**

Traceback (most recent call last):

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401

exec('\n'.join(lines[:-1]), glb)

~~~~~

File "<string>", line 4, in <module>

NameError: name 'df' is not defined

Cell 76: ■ Markdown

- Observación principal: los tipos de entrenamiento más frecuentes son Strength, Cardio, Yoga y HIIT; ambos géneros comparten el mismo top, con mayor conteo en hombres para cada categoría.
- Interpretación: las preferencias son similares entre géneros (mismas actividades dominantes) — las diferencias son de magnitud (más hombres en el dataset o más participación masculina en esas actividades), no de tipo de actividad.
- matiz: el dataset está dividido 50/50 en hombres y mujeres, por lo tanto la diferencia es real de preferencia dependiendo del sexo.

Cell 77: ■ Markdown

*****4 — Correlaciones entre variables demográficas y de estilo de vida*****

Por último, se examinan las relaciones entre las variables demográficas principales (`Gender`, `Height (m)`, `Weight (kg)`, `Age`) y el resto de variables numéricas del dataset mediante una **matriz de correlación**.

El objetivo es detectar posibles patrones entre el perfil físico de los individuos y otros indicadores (por ejemplo, gasto energético o frecuencia de entrenamiento).

Se genera un mapa de calor (*heatmap*) que permite visualizar rápidamente las correlaciones más fuertes y, posteriormente, se muestran las **10 correlaciones más altas** para cada variable analizada.

Esto facilita identificar asociaciones relevantes que podrían ser exploradas en análisis posteriores (por ejemplo, relaciones entre peso y gasto calórico o entre edad y nivel de actividad).

Cell 78: ■ Code

```
df['male'] = df['Gender'].map({'Male': 1, 'Female': 0}).fillna(-1)

# Variables de interés a comparar
features = ['male', 'Height (m)', 'Weight (kg)', 'Age']
features = [f for f in features if f in df.columns]

# Seleccionar columnas numéricas disponibles
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()

# Matriz de correlación completa entre numéricos
corr = df[numeric_cols].corr()

# Sub-matriz: filas = nuestras features, columnas = todas las numéricas
corr_sub = corr.loc[features, :]

# Heatmap (features vs resto de numéricas)
```

```
plt.figure(figsize=(14, len(features)*0.8 + 2))
sns.heatmap(corr_sub, annot=True, fmt=".2f", cmap='coolwarm', vmin=-1, vmax=1,
linewidths=.5, cbar_kws={"shrink": .75})
plt.title("Correlación: Gender/Height/Weight/Agevs columnas numéricas")
plt.tight_layout()
plt.show()

df.drop(columns=['male'])
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 1, in <module>
NameError: name 'df' is not defined
```

Cell 79: ■ Markdown

Relaciones principales

- male se asocia positivamente con Height, Weight y Water_Intake (indicando diferencias de tamaño/masa e hidratación entre géneros); también correlaciones moderadas con BMI.
- Weight está fuertemente correlacionado con BMI y lean_mass_kg (alta colinealidad entre medidas antropométricas).
- Height muestra correlaciones moderadas con Weight (más alto → mayor peso).
- Fat_Percentage se asocia negativamente con male (los hombres del dataset suelen tener menor %grasa) y tiene correlaciones negativas moderadas con algunas variables de rendimiento.
- Age presenta correlaciones débiles con la mayoría de variables numéricas (poca señal lineal en este mapa).

Cell 80: ■ Markdown

8■■ ****Modelado predictivo:** estimación de calorías quemadas**

En esta fase, se desarrolla un modelo predictivo cuyo objetivo es **estimar la cantidad de calorías quemadas (`Calories\_Burned`)** durante una sesión de entrenamiento a partir de distintos factores fisiológicos, demográficos y de rendimiento.

El gasto calórico depende de múltiples variables: edad, peso, frecuencia cardíaca, duración e intensidad del ejercicio, entre otras.

Mediante el uso de técnicas de aprendizaje supervisado (regresión), se busca construir un modelo que sea capaz de **predecir de forma aproximada las calorías quemadas** en función de dichas variables, y

así comprender cuáles son los factores que más influyen en el rendimiento energético.

Para ello, se seleccionan como variables explicativas (`features`) aquellas que, por su naturaleza, tienen relación directa con el gasto calórico:

- **Edad (`Age`)**: influye en el metabolismo y en la eficiencia energética.
- **Peso (`Weight (kg)`) y Altura (`Height (m)`)**: determinan la masa corporal total y el gasto energético basal.
- **Frecuencias cardíacas (`Avg\_BPM`, `Max\_BPM`, `Resting\_BPM`)**: reflejan el esfuerzo e intensidad durante el entrenamiento.
- **Duración de la sesión (`Session\_Duration (hours)`)**: influye directamente en el volumen total del esfuerzo.
- **Tipo de entrenamiento (`Workout\_Type`)**: distingue entre modalidades (por ejemplo, fuerza, cardio o HIIT) que tienen diferentes demandas energéticas.
- **Intensidad relativa (`Relative\_Intensity`)**: mide la proporción de esfuerzo realizado respecto a la capacidad máxima del individuo.
- **Nivel de experiencia (`Experience\_Level`)**: puede afectar la eficiencia con la que se realiza el ejercicio.
- **Carga semanal (`Weekly\_training\_load`)**: combina frecuencia y duración para reflejar el volumen total de entrenamiento.

La variable objetivo (`target`) será `Calories\_Burned`, que representa el **total de calorías consumidas en una sesión**.

Antes de entrenar el modelo, se realiza una **preparación del dataset** que incluye:

1. **Selección de variables relevantes.**
2. **Codificación de variables categóricas** (como `Workout\_Type`).
3. **División en conjuntos de entrenamiento y prueba** para evaluar el rendimiento del modelo.
4. **Escalado de las variables numéricas** para normalizar los rangos y mejorar la estabilidad del modelo.

A continuación, se muestra el proceso de preparación de los datos previo al entrenamiento.

Cell 81: ■ Code

```
# 1. Seleccionar variables relevantes
features = [
    'Age', 'Weight (kg)', 'Height (m)', 'Avg_BPM', 'Max_BPM', 'Resting_BPM',
    'Session_Duration (hours)', 'Experience_Level',
    'Weekly_training_load', 'Workout_Type'
]
```

```

target = 'Calories_Burned'

X = df[features].copy()
y = df[target]

# 2. Convertir variables categóricas (como Workout_Type) a numéricas
X = pd.get_dummies(X, columns=['Workout_Type'], drop_first=True)

# 3. Dividir en conjunto de entrenamiento y prueba
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 4. Escalar variables numéricas (importante para regresiones)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 9, in <module>
NameError: name 'df' is not defined

```

Cell 82: ■ Markdown

■ 8.3 Regresión Lineal Simple

Antes de aplicar modelos complejos, se comienza con una **regresión lineal simple** para comprender la relación directa entre una única variable predictora y la variable objetivo.

En este caso, se analiza la relación entre la **duración de la sesión de entrenamiento** ('Session_Duration (hours)') y las **calorías quemadas** ('Calories_Burned').

Este tipo de modelo permite observar si existe una tendencia lineal clara: es decir, si al aumentar la duración del entrenamiento, el gasto calórico aumenta de forma proporcional.

Cell 83: ■ Code

```

# Entrenamiento con una sola variable
X_simple = df[['Session_Duration (hours)']]

```

```

y = df['Calories_Burned']

model_simple = LinearRegression()
model_simple.fit(X_simple, y)

# Predicciones
y_pred_simple = model_simple.predict(X_simple)

# Visualización
sns.regplot(x='Session_Duration (hours)', y='Calories_Burned', data=df,
line_kws={'color':'red'})
plt.title('Regresión Lineal Simple: Duración vs Calorías Quemadas')
plt.show()

# Coeficiente de determinación
print(f"R²: {model_simple.score(X_simple, y):.3f}")

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401, in
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined

```

Cell 84: ■ Markdown

■ 8.4 Regresión Lineal Múltiple

Una vez analizada la relación individual entre la duración de la sesión y el gasto calórico, se amplía el modelo para incorporar **todas las variables predictoras relevantes**.

El objetivo es construir un modelo que pueda estimar las calorías quemadas (`Calories\_Burned`) en función de variables demográficas, fisiológicas y de entrenamiento, incluyendo:

- Peso, altura y edad.
- Frecuencias cardíacas promedio, máxima y en reposo.
- Duración e intensidad de la sesión.
- Nivel de experiencia y tipo de entrenamiento.

Este enfoque multivariable permite capturar mejor la complejidad del fenómeno, ya que las calorías quemadas dependen de múltiples factores simultáneamente.

A continuación, se entrena el modelo y se evalúan sus resultados utilizando métricas estándar:

- **MAE (Error Absoluto Medio)**: promedio de los errores en unidades originales.

- **RMSE (Raíz del Error Cuadrático Medio)**: penaliza errores grandes.
- **R² (Coeficiente de determinación)**: indica qué porcentaje de la variabilidad en `Calories\_Burned` es explicado por el modelo.

Cell 85: ■ Code

```

model_multi = LinearRegression()
model_multi.fit(X_train_scaled, y_train)

# Predicciones
y_pred_multi = model_multi.predict(X_test_scaled)

# Métricas (compatibles con todas las versiones)
mae = mean_absolute_error(y_test, y_pred_multi)

rmse = np.sqrt(mean_squared_error(y_test, y_pred_multi))

r2 = r2_score(y_test, y_pred_multi)

print(f"MAE: {mae:.3f}")
print(f"RMSE: {rmse:.3f}")
print(f"R²: {r2:.3f}")

# Visualización: predicciones vs valores reales
plt.figure(figsize=(6, 5))
sns.scatterplot(x=y_test, y=y_pred_multi, alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("Valores reales (Calories_Burned)")
plt.ylabel("Predicciones")
plt.title("Predicciones vs Valores reales (Regresión múltiple)")
plt.show()

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'X_train_scaled' is not defined

```

Cell 86: ■ Markdown

- Rendimiento: el modelo lineal explica la mayor parte de la variabilidad observada y presenta un error razonable, por lo que captura bien la relación entre las features seleccionadas y Calories_Burned.
- Interpretación práctica: las variables de volumen e intensidad (Session_Duration, Calories_per_hour / proxies de intensidad, Workout_Frequency / Weekly_training_load) son las que aportan más señal —la duración y la intensidad explican gran parte del gasto calórico.

Cell 87: ■ Markdown

9■■ ****Regularización (Ridge y Lasso) + Validación cruzada****

En este apartado aplicamos técnicas de ****regularización**** para mejorar la capacidad predictiva del modelo lineal y reducir el riesgo de ****sobreajuste****.

Las regresiones ****Ridge**** y ****Lasso**** añaden un término de penalización a los coeficientes del modelo, controlando su magnitud:

- ****Ridge (L2):**** penaliza la suma de los cuadrados de los coeficientes. Tiende a reducir su tamaño sin anularlos.
- ****Lasso (L1):**** puede reducir algunos coeficientes exactamente a cero, actuando como un método de ***selección automática de variables***.

Ambos modelos se evaluarán mediante ****validación cruzada (cross-validation)**** para obtener una estimación más robusta del rendimiento general del modelo.

Cell 88: ■ Code

```
from sklearn.linear_model import RidgeCV, LassoCV

# Ridge Regression (regulariza la magnitud de los coeficientes)
ridge = RidgeCV(alphas=[0.1, 1.0, 10.0, 100.0], cv=5)
ridge.fit(X_train_scaled, y_train)

# Lasso Regression (puede eliminar variables poco relevantes)
lasso = LassoCV(alphas=[0.001, 0.01, 0.1, 1.0], cv=5, max_iter=5000,
random_state=42)
lasso.fit(X_train_scaled, y_train)

# Evaluación
models = {
    "Linear": model_multi,
    "Ridge": ridge,
    "Lasso": lasso
}
```

```

results = []
for name, m in models.items():
    y_pred = m.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    results.append([name, r2, rmse])

results_df = pd.DataFrame(results, columns=["Modelo", "R²", "RMSE"])
display(results_df)

print(f"Mejor alpha Ridge: {ridge.alpha_}")
print(f"Mejor alpha Lasso: {lasso.alpha_}")

# Visualizar coeficientes
coef_df = pd.DataFrame({
    'Variable': X_train.columns,
    'Ridge': ridge.coef_,
    'Lasso': lasso.coef_
}).set_index('Variable')

coef_df.plot(kind='bar', figsize=(12,5), title='Comparativa de Coeficientes Ridge vs Lasso')
plt.tight_layout()
plt.show()

```

Error:

```

Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^
  File "<string>", line 5, in <module>
NameError: name 'X_train_scaled' is not defined

```

Cell 89: ■ Markdown

Los tres modelos presentan prácticamente el mismo rendimiento ($R^2 \approx 0.9635$, $RMSE \approx 95.34$), por lo que la regularización no aporta mejoras significativas respecto al modelo lineal. Esto indica baja probabilidad de sobreajuste y una relación bien capturada por el modelo base. Ridge mantiene todos los coeficientes y Lasso apenas reduce algunos, lo que sugiere que ninguna variable es claramente prescindible en esta predicción.

Cell 90: ■ Markdown

■ ****Árboles de regresión y ajuste de hiperparámetros****

En esta sección entrenamos un ****árbol de regresión****, un modelo no lineal que divide los datos en ramas según las variables que mejor predicen el valor objetivo, el valor objetivo elegido será Resting_BPM, es un fuerte indicador de la salud del usuario pero tiene relaciones más complejas que calories_burned, por eso lo probamos en este modelo y no el anterior.

Los árboles permiten capturar ****relaciones no lineales**** entre las variables y detectar interacciones entre características.

Se construirá un modelo base y luego se aplicará un proceso de ****ajuste de hiperparámetros (GridSearchCV)**** para optimizar parámetros clave como:

- Profundidad máxima del árbol (`max\_depth`)
- Mínimo de muestras por división (`min\_samples\_split`)
- Mínimo de muestras por hoja (`min\_samples\_leaf`)

Finalmente, se analizará la ****importancia de las variables**** y se evaluará la mejora obtenida tras el ajuste.

Cell 91: ■ Code

```
# Target = Resting_BPM
y_hrr = df['Resting_BPM']

# Selección explícita de features relevantes (evitar columnas derivadas que
incluyen Resting_BPM)
features_hrr = [
    'Age', 'Weight (kg)', 'Height (m)', 'BMI', 'Fat_Percentage',
    'Workout_Frequency', 'Session_Duration (hours)', 'Calories_Burned',
    'Calories_per_hour', 'Experience_Level',
    'Weekly_training_load', 'Workout_Type'
]

# Filtrar solo las columnas que existen en el DataFrame
features_present = [f for f in features_hrr if f in df.columns]
X_model = df[features_present].copy()

# Codificar categóricas si existen
if 'Workout_Type' in X_model.columns:
    X_model = pd.get_dummies(X_model, columns=['Workout_Type'], drop_first=True)
```

```

# Split (mismo random_state para comparabilidad)
X_train_hrr, X_test_hrr, y_train_hrr, y_test_hrr = train_test_split(X_model, y_hrr,
test_size=0.2, random_state=42)

# Árbol base
tree_base = DecisionTreeRegressor(random_state=42)
tree_base.fit(X_train_hrr, y_train_hrr)

r2_train_base = tree_base.score(X_train_hrr, y_train_hrr)
r2_test_base = tree_base.score(X_test_hrr, y_test_hrr)
rmse_base = np.sqrt(mean_squared_error(y_test_hrr, tree_base.predict(X_test_hrr)))

print(f"Árbol base -> R² train: {r2_train_base:.3f}, R² test: {r2_test_base:.3f}, RMSE: {rmse_base:.2f}")

# Grid Search para optimización
params = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 5, 10]
}

grid_tree = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid=params,
cv=5, scoring='r2', n_jobs=-1)
grid_tree.fit(X_train_hrr, y_train_hrr)

best_tree = grid_tree.best_estimator_
print("\nMejores hiperparámetros:", grid_tree.best_params_)

# Evaluación
r2_train = best_tree.score(X_train_hrr, y_train_hrr)
r2_test = best_tree.score(X_test_hrr, y_test_hrr)
rmse = np.sqrt(mean_squared_error(y_test_hrr, best_tree.predict(X_test_hrr)))

print(f"Árbol optimizado -> R² train: {r2_train:.3f}, R² test: {r2_test:.3f}, RMSE: {rmse:.2f}")

# Importancia de variables
importances = pd.DataFrame({
    'Variable': X_model.columns,
    'Importancia': best_tree.feature_importances_
})

```

```
}).sort_values('Importancia', ascending=False)

sns.barplot(x='Importancia', y='Variable', data=importances.head(10))
plt.title("Importancia de variables (Árbol optimizado) - Target: Resting_BPM")
plt.show()
```

Error:

```
Traceback (most recent call last):
  File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
  File "<string>", line 2, in <module>
NameError: name 'df' is not defined
```

Cell 92: ■ Markdown

El árbol optimizado indica que el ritmo cardíaco en reposo (Resting_BPM) está fuertemente relacionado con la frecuencia/carga de entrenamiento: variables como `Weekly\_training\_load` y `age` aparecen entre las más relevantes para predecir Resting_BPM. Esto sugiere que quienes entrenan con mayor regularidad presentan diferencias sistemáticas en Resting_BPM, coherente con efectos sobre la condición cardiovascular.

Cell 93: ■ Markdown

1■■■1■■■ ****Random Forest vs Gradient Boosting****

En este bloque comparamos dos métodos avanzados de *ensemble learning* que combinan múltiples árboles de decisión para lograr predicciones más estables y precisas.

- ****Random Forest:**** construye muchos árboles independientes sobre subconjuntos aleatorios de datos y promedia sus predicciones. Reduce el sobreajuste y mejora la generalización.
- ****Gradient Boosting:**** entrena árboles de forma secuencial, donde cada nuevo árbol corrige los errores de los anteriores, logrando un modelo más afinado.

Ambos modelos se entrenarán con las mismas variables predictoras del modelo lineal, y se evaluarán mediante:

- Coeficiente de determinación (****R²****)
- Raíz del error cuadrático medio (****RMSE****)
- Validación cruzada (****CV****)

Finalmente, se comparará el rendimiento y la importancia de las variables en ambos métodos para determinar cuál explica mejor las calorías quemadas.

Cell 94: ■ Code

```

features_rb = [
    'Age', 'Weight (kg)', 'Height (m)', 'BMI', 'Fat_Percentage',
    'Workout_Frequency', 'Session_Duration (hours)', 'Experience_Level',
    'Weekly_training_load', 'Calories_per_hour', 'Calories_Burned'
]

# conservar solo las que existan
features_rb = [f for f in features_rb if f in df.columns]

X = df[features_rb].copy()

# codificar categóricas si aparece Workout_Type en el df original y no está en
features_rb

if 'Workout_Type' in df.columns and 'Workout_Type' not in X.columns:
X = pd.concat([X, pd.get_dummies(df['Workout_Type'], prefix='Workout',
drop_first=True)], axis=1)

y = df['Resting_BPM'].copy()

from sklearn.model_selection import train_test_split
# reemplaza/crea X_train, X_test, y_train, y_test para este objetivo
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Entrenamiento y evaluación con RF / GB (hiperparámetros conservadores)
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

rf = RandomForestRegressor(n_estimators=200, max_depth=7, min_samples_leaf=5,
random_state=42, n_jobs=-1)
gb = GradientBoostingRegressor(n_estimators=200, learning_rate=0.05, max_depth=3,
random_state=42)

rf.fit(X_train, y_train)
gb.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
y_pred_gb = gb.predict(X_test)

print("Random Forest R²:", r2_score(y_test, y_pred_rf))
print("Gradient Boosting R²:", r2_score(y_test, y_pred_gb))

```

```

# Métricas
def evaluate_model(name, y_true, y_pred, model):
    r2 = r2_score(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    cv = cross_val_score(model, X_train, y_train, cv=5, scoring='r2').mean()
    return [name, r2, rmse, cv]

results = [
    evaluate_model("Random Forest", y_test, y_pred_rf, rf),
    evaluate_model("Gradient Boosting", y_test, y_pred_gb, gb)
]

results_df = pd.DataFrame(results, columns=["Modelo", "R² Test", "RMSE Test", "R² CV"])
display(results_df)

# ■ Comparación de R²
plt.figure(figsize=(6,4))
sns.barplot(x='Modelo', y='R² Test', data=results_df, palette=['forestgreen', 'orange'])
plt.title("Comparación de rendimiento: Random Forest vs Gradient Boosting (Resting_BPM)")
plt.ylabel("R² (test)")
plt.grid(axis='y', alpha=0.3)
plt.show()

# ■ Feature Importance (RF)
rf_importance = pd.DataFrame({
    'Variable': X_train.columns,
    'Importancia': rf.feature_importances_
}).sort_values('Importancia', ascending=False)

gb_importance = pd.DataFrame({
    'Variable': X_train.columns,
    'Importancia': gb.feature_importances_
}).sort_values('Importancia', ascending=False)

top_features = set(rf_importance.head(8)['Variable']) |
set(gb_importance.head(8)['Variable'])

```

```

comparison = pd.DataFrame({
    'Variable': list(top_features),
    'Random Forest': [rf_importance.set_index('Variable').loc[var, 'Importancia'] if
    var in rf_importance['Variable'].values else 0 for var in top_features],
    'Gradient Boosting': [gb_importance.set_index('Variable').loc[var, 'Importancia']
    if var in gb_importance['Variable'].values else 0 for var in top_features]
}).set_index('Variable')

comparison.plot(kind='barh', figsize=(8,5), color=['forestgreen', 'orange'])
plt.title("Importancia de variables: RF vs GB (Top 8) - Resting_BPM")
plt.xlabel("Importancia")
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()

```

Error:

Traceback (most recent call last):

```

File "c:\Users\rotap\.vscode\extensions\ganeshkumbhar.nb2pdf-1.1.9\scripts\nb2pdf.py", line 401
    exec('\n'.join(lines[:-1]), glb)
    ~~~~^~~~~~
File "<string>", line 7, in <module>
NameError: name 'df' is not defined

```

Cell 95: ■ Markdown

Qué nos dice:

- Hay señal útil: variables como Weekly_training_load, Weight, Session_Duration, Height, BMI y Age aportan información relevante (coherente clínicamente).
- Sin embargo, la capacidad predictiva es limitada: Resting_BPM parece depender de factores no presentes en las features (p. ej. genética, sueño, estrés, medicación, medidas continuas de actividad) o de ruido/variabilidad individual.
- RF supera a GB con estos datos y parámetros, pero la diferencia no es muy grande.

Conclusión final: los modelos ofrecen insights útiles a nivel poblacional sobre qué factores se asocian a Resting_BPM, pero no son suficientemente precisos para predicción individual. Mejoran la explicación parcial del fenómeno; para un modelo predictivo robusto se requieren más y/o mejores datos, validación por grupos y ajuste fino.

Cell 96: ■ Markdown

■ ****Conclusiones finales****

Cell 97: ■ Markdown

El análisis desarrollado abarca la limpieza del conjunto de datos, la generación de variables derivadas relevantes (HRR, Relative_Intensity, Calories_per_hour y Weekly_training_load), un estudio descriptivo y de correlaciones, y la evaluación de diversos modelos predictivos —tanto lineales como basados en árboles (Random Forest y Gradient Boosting)— para estimar Calories_Burned y Resting_BPM.

En cuanto a la calidad de los datos, se identificaron inconsistencias en las columnas relacionadas con la dieta (Calories, Carbs, Proteins, Fats y diet_type), por lo que fueron descartadas del análisis; no se detectaron valores nulos. Las variables derivadas resultaron coherentes y aportaron información útil: Weekly_training_load, Age, Weight, Session_Duration, BMI y Height se destacaron de forma recurrente como factores asociados a Resting_BPM.

En el modelado predictivo, el Random Forest explicó aproximadamente un 32 % de la varianza de Resting_BPM, mientras que el Gradient Boosting alcanzó cerca de 26 % . Estos resultados reflejan un rendimiento moderado, adecuado para capturar tendencias generales, aunque limitado para predicciones individuales de alta precisión.

En conjunto, los hallazgos confirman la existencia de patrones significativos entre las características de los deportistas y la frecuencia cardíaca en reposo, pero también evidencian que una parte sustancial de la variabilidad permanece sin explicar. Esto sugiere la influencia de factores externos no registrados —como la calidad del sueño, el nivel de estrés, la medicación o la genética—, así como variaciones individuales inherentes.